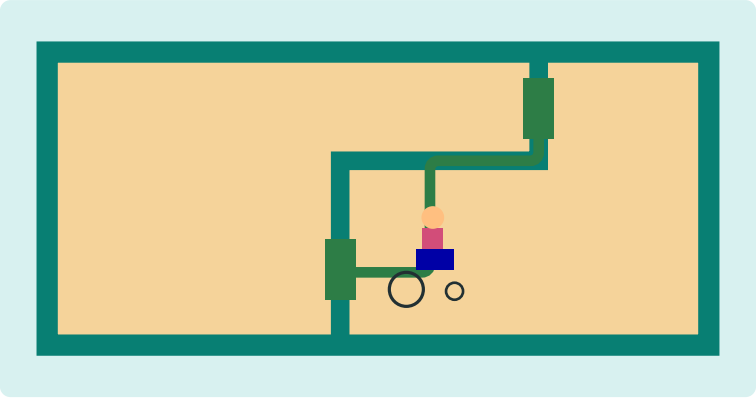
ACC Manual

How the Wheelchair Route Checker Works

From an IFC file to 2D plans, a 2.5D simulation, SHACL results, and an Ollama explanation

Accessibility Checking Assistant

July 2026



This manual describes the code that is in the project. It separates route calculation, rule checking, drawing, animation, and language generation so that each part is easy to identify.

Contents

1 The whole project in one sentence 3

2 Stage 1: Reading the IFC building 3

3 Stage 2: Turning IFC meaning into RDF 4

4 Stage 3: Finding which doors share a space 4

5 Stage 4: Building two occupancy grids 5

6 Stage 5: A* finds orthogonal routes 6

7 Stage 6: Measuring each route edge 8

8 Stage 7: SHACL checks the rules 9

9 Stage 8: Dijkstra joins route edges 9

10 Stage 9: Writing the application package 10

11 Stage 10: The Python web server 11

12 Stage 11: The 2D floor plan 11

13 Stage 12: The building-model view 12

14 Stage 13: The wheelchair simulation is 2.5D 12

15 Stage 14: Ollama explains; it does not judge 13

16 Which part answers which question? 14

17 Important limits 14

18 Source map for technical readers 15

19 Short recap 15

1 The whole project in one sentence

The program reads a digital building file, turns rooms and building parts into measurable data, finds routes between doors, checks those routes against accessibility rules, and shows the result in several views.

There is no single “magic algorithm.” Different jobs use different tools:

- **IfcOpenShell** reads geometry and IFC relationships.
- **IFCtoLBD** converts IFC meaning into RDF triples.
- **A*** finds a path across a room while avoiding blocked grid cells.
- **Dijkstra’s algorithm** joins door-to-door edges into shortest routes across a floor.
- **SHACL** checks measurements and assigns pass/fail reasons.
- **SVG** draws the 2D floor plan.
- **Three.js and WebGL** draw the building model and wheelchair simulation.
- **Ollama** writes a short explanation of results that already exist.

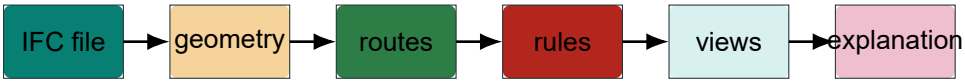


Figure 1: The six kinds of work. This is a labelled system strip, not the route-search graph.

2 Stage 1: Reading the IFC building

What goes in

An **IFC file** is a structured building model. It can contain walls, doors, rooms, stairs, ramps, storeys, names, identifiers, placements, and shapes. The program does not treat it as a picture. It reads objects and relationships.

What technology does the work

IfcOpenShell opens the IFC file. Its geometry engine creates a shape for each useful object. The project calculates an axis-aligned bounding box: the smallest box, aligned with the X, Y, and Z axes, that encloses the shape. It also records the centre, size, IFC type, storey, and GlobalId.

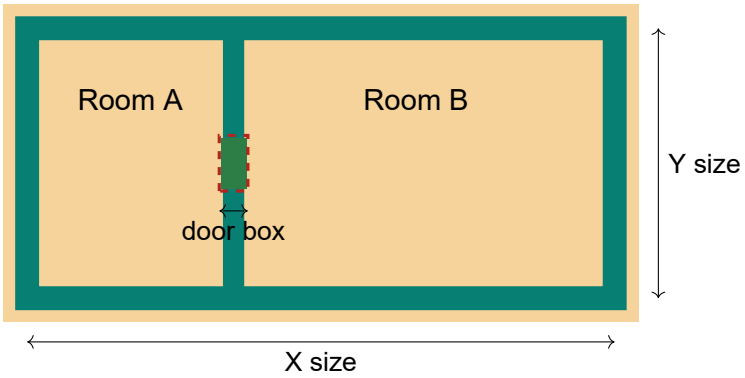


Figure 2: Plan-view drawing of two spaces, a wall and one door bounding box.

Why storeys matter

Routes should connect doors on the same floor. The code compares storey data and elevation. Objects from a roof or another floor must not silently become floor obstacles. The simulation later flattens a selected floor into a floor slice, but the original geometry remains available for measurements.

3 Stage 2: Turning IFC meaning into RDF

Geometry says where an object is. It does not always say what that object means in a way that a rule engine can query easily. **IFCtoLBD 2.43.4** converts IFC information into **RDF**.

RDF stores small facts called **triples**: subject, predicate, object. For example:

```
Door-18 hasStorey GroundFloor
```

The converter is Java software. The project uses its pinned Java libraries so that the same converter version is used on another computer. The output is a Turtle file named `raw_lbd_graph.ttl`. The program parses this file with **RDFLib**.

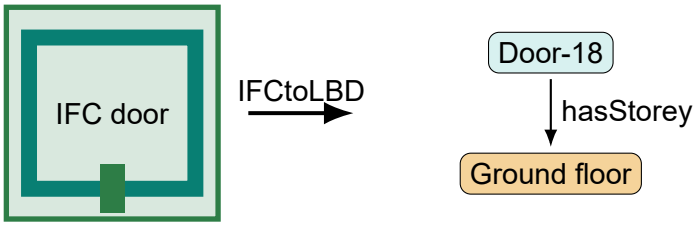


Figure 3: The building object remains geometry; RDF adds machine-readable meaning.

IFCtoLBD does not find wheelchair routes. It prepares semantic building facts. Route geometry is calculated separately by Python code.

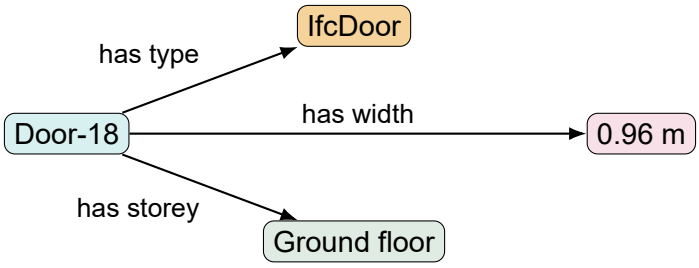


Figure 4: One RDF subject can connect to several facts. This is a fact drawing, not a route graph.

4 Stage 3: Finding which doors share a space

The route builder reads `IfcRelSpaceBoundary`. This IFC relationship says that an element touches or bounds a space. When a space has two or more usable doors, the program considers door pairs inside that space.

Lift doors and objects marked as unsuitable route doors are excluded. Duplicate doors are removed. The result is not yet a path. It is only a list of door pairs that may be connectable through the same room or corridor.

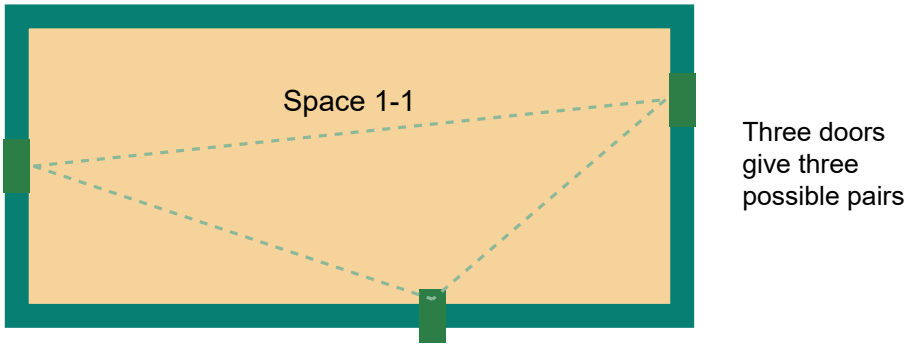


Figure 5: Candidate pairs are made only between doors that belong to the same space boundary set. Dashed lines here mean “pair to test,” not final routes.

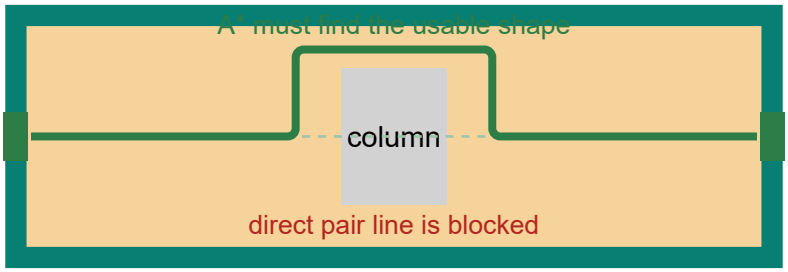


Figure 6: A door pair only says what should be connected. It does not assume that the straight line is safe.

5 Stage 4: Building two occupancy grids

The final implementation uses two four-direction grids because it solves two different route questions.

The **door-route grid** is built inside each shared IFC space. Each cell represents **0.10 m by 0.10 m**. Preprocessing uses this grid to calculate the stored door-to-door RouteEdges. Walls, stairs, columns and other hard obstacles are marked as blocked from their IFC-derived bounding boxes. These boxes are expanded by **0.38 m** for the visual door-route clearance used by this project.

The **interactive point-route grid** covers each complete floor at **0.01 m by 0.01 m** resolution. It blocks walls, columns, stairs, inaccessible ramps, doors narrower than 0.90 m, and spaces narrower than the 1.50 m route-width requirement. A 0.90 m wheelchair envelope has a 0.45 m radius. After the 0.005 m geometric tolerance is applied, wall and hard-obstacle rectangles are expanded by **0.445 m** around the route centre.

Doors need controlled openings. The program splits wall rectangles at accessible doors and creates a portal only where the door is wide enough. This prevents a small gap beside a door from becoming a false route through the wall.

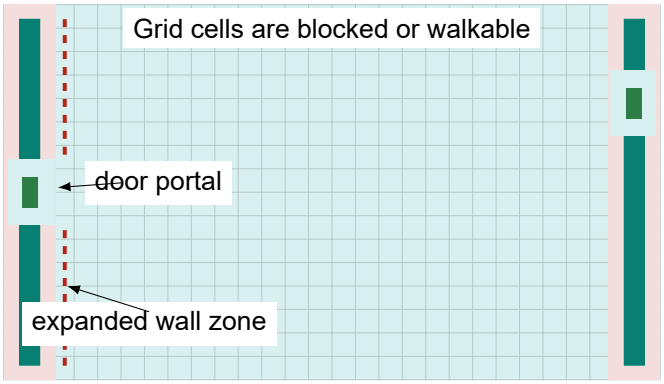


Figure 7: A real occupancy-grid drawing: blocked margins remain beside the wall, while the door portal is opened.

The point-route grid is divided into **5 m by 5 m tiles**. Each 500 by 500 cell tile is packed into bits, compressed with zlib, and stored under `navigation/tiles`. The server loads only tiles needed by the selected floor and route, checks each tile's SHA-256 digest, and keeps at most 64 decoded tiles in memory.

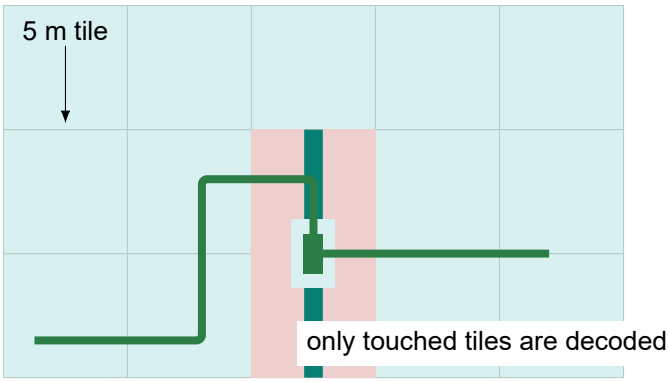


Figure 8: The 0.01 m floor grid is streamed as 5 m tiles. The green route crosses the wall only through the accessible door portal.

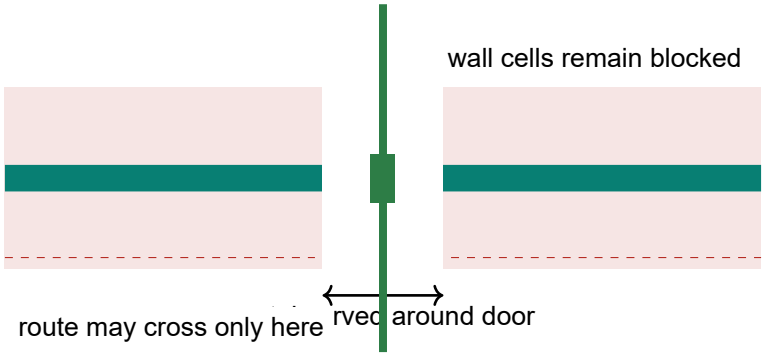


Figure 9: Close-up of a door portal. The opening is local; it does not erase the rest of the wall barrier.

6 Stage 5: A* finds orthogonal routes

A* (A-star) searches an occupancy grid. For a stored RouteEdge it starts at one door and tries to reach the other door. For an interactive point route it starts at the selected point and tries to reach the selected

destination. Every move costs one grid step. The estimate to the target is the Manhattan distance:

$$h = |x - x_{target}| + |y - y_{target}|.$$

The program permits only four moves: right, left, up, and down. Diagonal moves are absent. Therefore, grid movement is orthogonal: 0, 90, or 180 degrees.

A* chooses the cell with the smallest value of:

$$f = g + h,$$

where g is the cost already travelled and h is the estimated cost still needed.

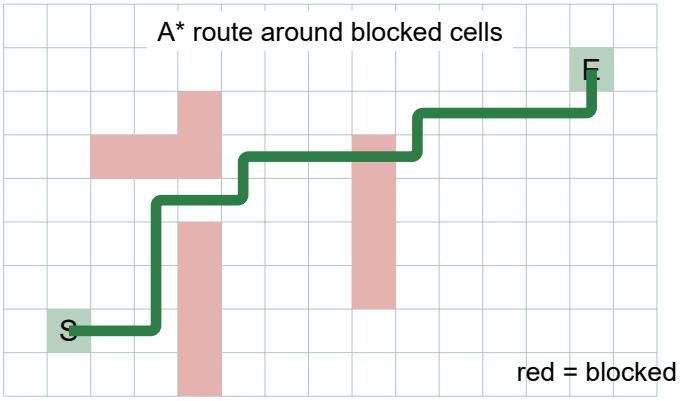


Figure 10: A* does not “see” the rendered wall mesh. It searches blocked and free cells derived from bounding boxes.

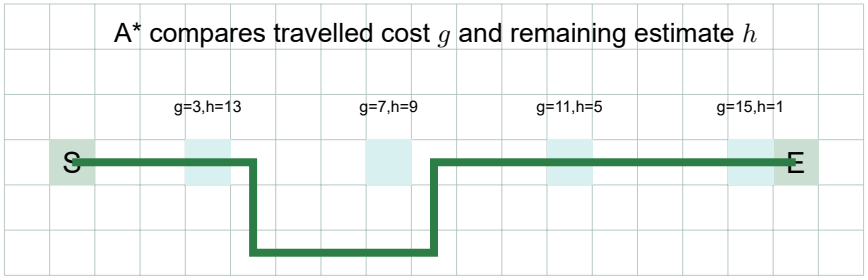


Figure 11: Cost labels on selected search cells. Lower $g + h$ values are explored first.

Exact door endpoints

Grid cells rarely sit exactly at a door centre. The code adds orthogonal endpoint connectors between the exact door centre and the first or last grid point. It then removes duplicate points, simplifies points that lie on the same straight segment, and densifies the route again.

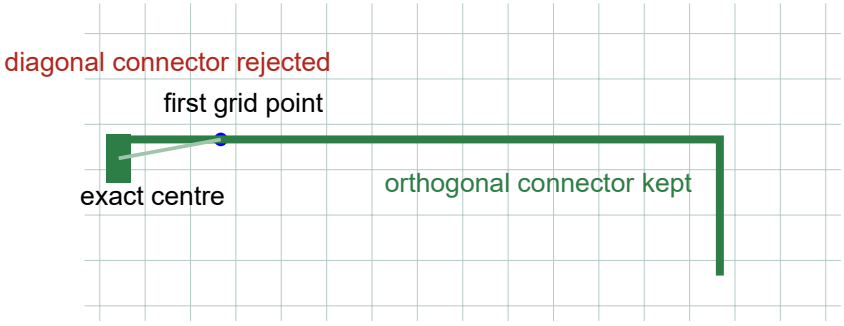


Figure 12: The endpoint connector starts at the actual door centre without introducing a diagonal segment.

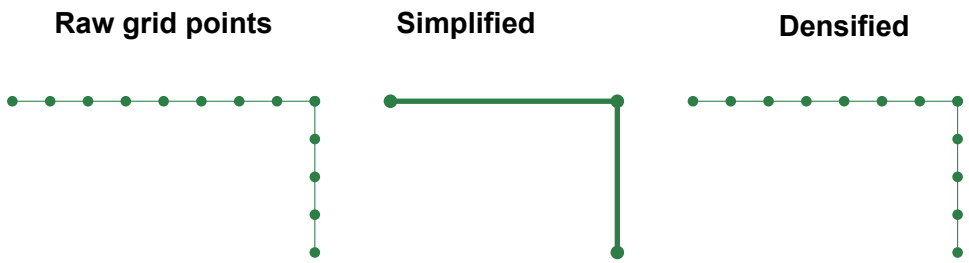


Figure 13: Simplification stores clean corners; densification restores closely spaced checking and animation points.

The strict final check

After construction, every dense point is checked against the occupancy grid. If the route is not clear, preprocessing stops for that pair with an error. The program does not keep a route merely because it has fewer collisions.

Interactive point routes apply the same rule when the request arrives. The backend maps both selected coordinates to stored navigation cells, loads the necessary tiles, runs four-direction A*, restores the exact clicked endpoints with orthogonal connectors, and audits the complete path. A route is green only when it reaches the requested destination, remains collision-free, and contains no diagonal segment.

If the destination is blocked or outside the accessible walking area, the backend may return a **red candidate**. This is not presented as an accessible route. It stays inside the start point's reachable component and ends at the last collision-free point before the obstruction. If the start itself is blocked, the result contains no path and the wheelchair Play button is disabled.

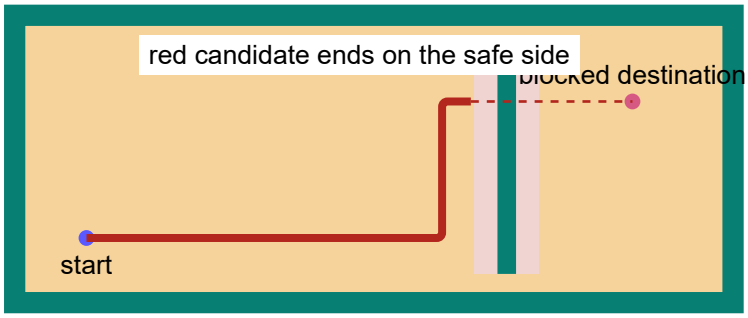


Figure 14: A blocked destination never turns a partial candidate into a green route. The dashed part is not travelled or returned as safe.

7 Stage 6: Measuring each route edge

Each successful A* path becomes a **RouteEdge**. It stores the two door IDs, a polyline, distance, measurements, status, and failure reasons.

Measurements include:

- minimum door width;
- route clear width;
- turning space when the path bends;
- whether the route intersects a stair;
- ramp slope and usable ramp width when applicable;
- whether the route remains orthogonal.

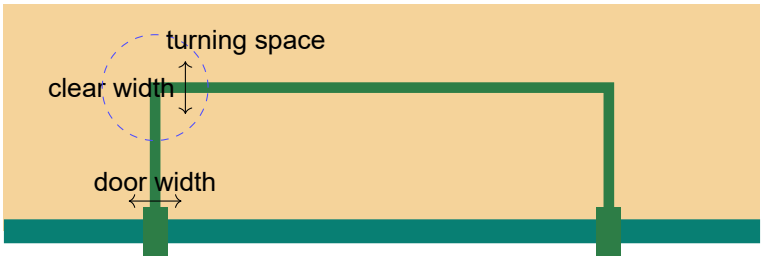


Figure 15: Route measurements are numerical facts. The drawing is only an explanation of those facts.

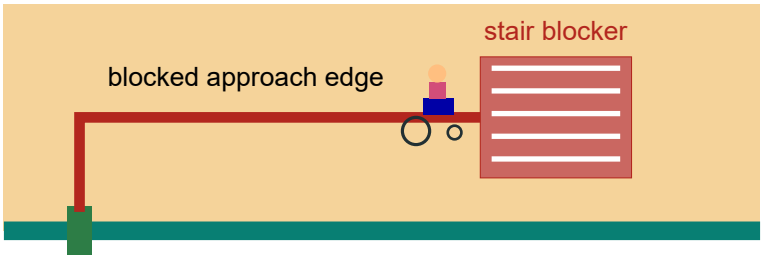


Figure 16: A stair-approach edge is deliberately visible and failed. The chair stops before the stair.

Special blocked stair edges

The project also creates orthogonal approach edges from relevant doors toward stairs. These edges exist so the interface can show exactly where a wheelchair route becomes blocked by a stair. They are not successful accessible routes.

8 Stage 7: SHACL checks the rules

SHACL means Shapes Constraint Language. It is a rule language for RDF graphs. Python adds route measurements to the RDF graph, and **pySHACL** runs the shapes in `rules/accessibility_rules.shacl.ttl`.

SHACL checks facts such as door width, route width, turning space, stair intersection, ramp slope, and ramp width. A failed shape produces a rule identifier. The program maps that identifier to a reason such as `door_width`, `route_width`, `turning_space`, or `stair_block`.

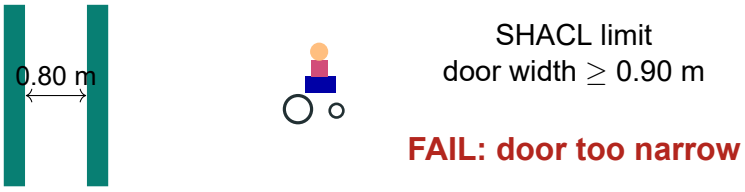


Figure 17: Example of a rule decision. SHACL compares stored numbers; it does not inspect pixels.

Ollama does not decide whether a route passes. The SHACL report and route measurements are the evidence.

9 Stage 8: Dijkstra joins route edges

A* works inside one shared space and creates one edge between two doors. A floor may contain many rooms, so the program needs a second algorithm.

Dijkstra's algorithm treats doors as graph nodes and RouteEdges as graph links. The link weight is route distance in metres. Starting at a door, it repeatedly selects the nearest unreached door. This produces the shortest known route to every reachable door.

When the accessible index is built, failed edges are removed first. Dijkstra then returns only paths made from passing edges.

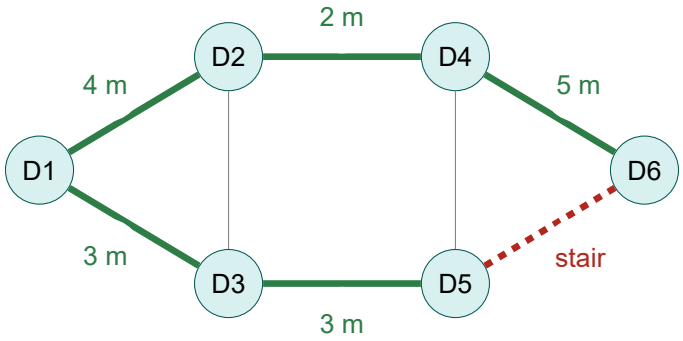


Figure 18: A real door graph drawing. Dijkstra can use green passing edges. A stair-blocked edge is excluded from the accessible graph.

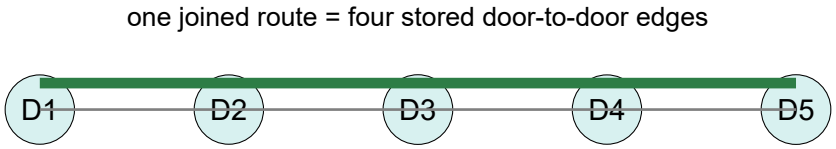


Figure 19: The browser may animate a route made from several RouteEdge polylines joined in Dijkstra order.

10 Stage 9: Writing the application package

Preprocessing writes a folder that the web server can load. Important files are:

`app_data.json`
: elements, floors, route edges, issues, routes by door, and accessible routes by door;

`raw_lbd_graph.ttl`
: RDF created by the pinned IFCtoLBD converter;

`lbd_graph.ttl`
: RDF plus calculated geometry and route facts;

`shacl_report.ttl`
: the full SHACL validation report;

`route_model.glb`
: compact building boxes and route polylines for the building-model view;

`route_graph.bin`
: optional fast binary copy of route edges;

`navigation/index.json`
: version, grid rules, floor bounds, tile metadata and integrity hashes;

`navigation/tiles/*.nav`
: compressed walkability tiles for interactive point routing.

The door graph and navigation tiles are precomputed. The browser does not rerun A* every animation frame. An interactive point request runs A* once on the precomputed 0.01 m floor cells; the resulting path is then drawn or animated.

11 Stage 10: The Python web server

`server.py` uses Python's HTTP server. It serves HTML, CSS, JavaScript, the package files, model-library data, door-route queries, point-route requests, assistant questions, project-service controls, and the Ollama restart request.

The server is also responsible for choosing a selected model package. Browser requests such as `/api/data` return JSON. A `POST /api/point-route` request contains the floor name and two IFC X/Y coordinate pairs. The deterministic backend returns the status, reason, polyline, distance, tile count and final audit. Rendering then works in the browser.

12 Stage 11: The 2D floor plan

The 2D plan is drawn as **SVG**, which means Scalable Vector Graphics. JavaScript selects elements and route edges for one floor, calculates floor bounds, and maps world X/Y positions into SVG coordinates.

Rooms, corridors, walls, doors, stairs, ramps, issues, and route polylines are separate SVG shapes. Green means passing; red means blocked. Route filtering changes what is visible, not how a stored door route was calculated.

Point-to-point mode adds two clicks to the same plan. The first click selects the start and the second selects the destination. The browser sends their IFC X/Y coordinates to the server. A green orthogonal line means the backend found and audited a complete route. A red line is only the reachable candidate: it ends at the last safe cell before the reported obstruction. The coloured dots mark the exact selected points; they do not enlarge the routing obstacles.

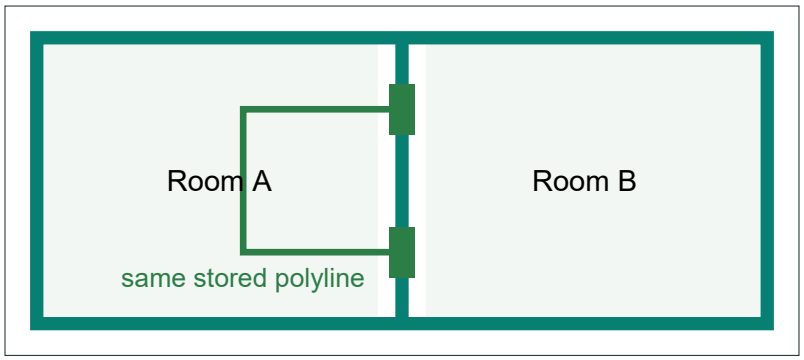


Figure 20: The 2D view projects a route onto a flat SVG plan.

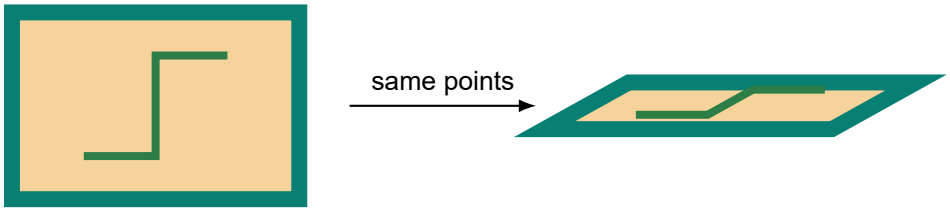


Figure 21: The 2D and simulation views use the same route response and draw its coordinates differently.

13 Stage 12: The building-model view

The building-model tab uses **Three.js**, a JavaScript 3D library, and **WebGL**, the browser's graphics system. **GLTFLoader** loads `route_model.glb`. A perspective camera, lights, edge lines, door markers, ray casting, and orbit controls make the model inspectable.

This view is for examining the package. It is not the place where A* runs. Selecting a door asks the server for routes and draws stored route lines above the model.

14 Stage 13: The wheelchair simulation is 2.5D

The simulation uses 3D meshes, lighting, shadows and a camera, but it intentionally presents one flat floor slice. That is why **2.5D** is the most accurate name.

For the selected floor, JavaScript builds boxes for walls, doors and blockers. It uses one uniform scale and the same floor transform for every element and every route point. The X/Y route coordinates become the horizontal X/Z scene coordinates. Route height becomes one explicit floor-surface height.

Figure 22: A cutaway drawing of the 2.5D floor slice. Geometry and route share one transform and one floor level.

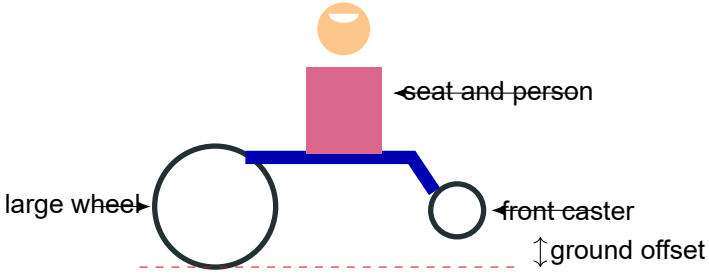


Figure 23: The wheelchair sprite is assembled from 3D primitives. Its lowest mesh point is placed on the floor surface.

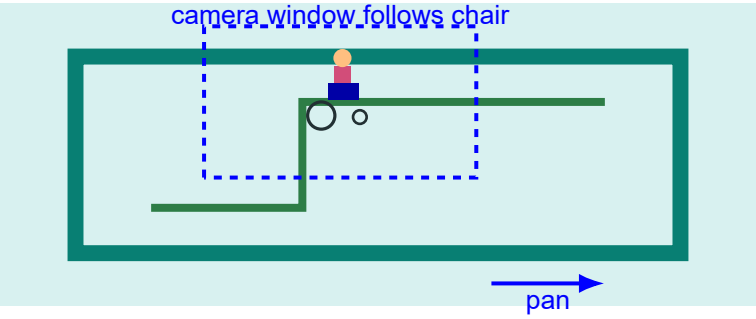


Figure 24: Panning moves the camera view. It does not move walls, change route coordinates, or rerun A*.

Animation

The chair is built from Three.js primitives: torus wheels, boxes, cylinders and spheres. The program calculates its lowest mesh point so the tyres touch the floor. During animation, it interpolates along the stored polyline and turns the chair toward the next point.

For a failed stored stair edge, the chair stops at the configured blocked point, waits briefly, and then advances to the next route. In point-to-point mode, a clear route makes one complete pass from the selected start to the selected destination. A blocked request uses the red reachable candidate and stops at its final safe point. If the selected start itself is blocked, there is no safe candidate to animate. This movement is presentation logic. It does not change the deterministic route result or the SHACL result.

The thin lines are the other stored floor RouteEdges. They provide context. The thick green or red line is the current animated route.

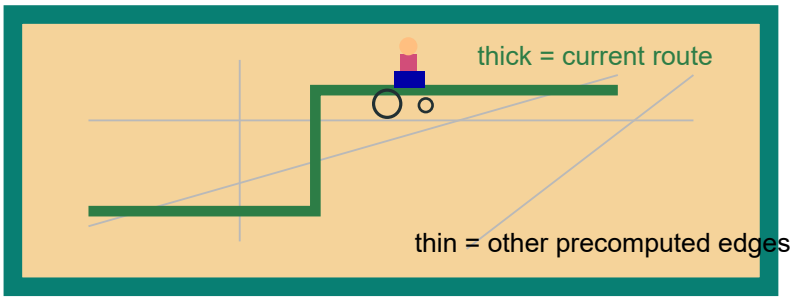


Figure 25: Route-line meaning in the wheelchair simulation.

15 Stage 14: Ollama explains; it does not judge

The Check Results page can send a question and a small block of validated facts to a local Ollama model. The model writes a short explanation. The default preferred model is `qwen3:8b` when installed.

The prompt tells the model to use only detected issue types, affected elements, failed routes, and floors with failures. Point-to-point explanations receive a separate structured fact set containing the selected coordinates, deterministic status, reason, distance, width and audit result. Ollama may reword those facts, but it cannot change a blocked result to clear, invent an elevator or ramp, or replace the backend reason. If Ollama is unavailable, the application returns a deterministic explanation from the same checked data.

The Restart Ollama button is maintenance code. It checks port 11434, refuses to kill an unrelated process, stops installed Ollama processes, starts one clean service, waits for the API, and warms the selected model.

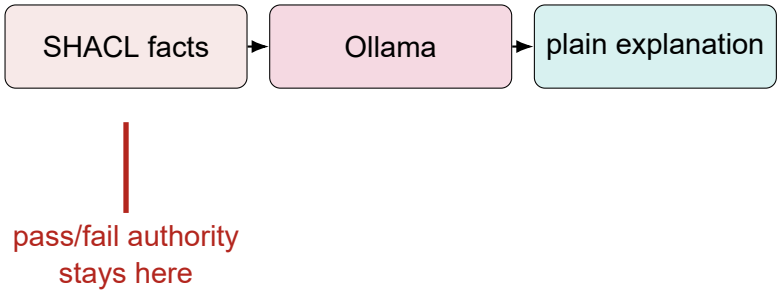


Figure 26: The language model changes wording, not route geometry or compliance status.

16 Which part answers which question?

Question	Responsible part	Not responsible
Where are the walls and doors?	IfcOpenShell geometry and bounding boxes	Ollama
What touches a space?	IFC relationships, especially IfcRelSpaceBoundary	SVG
How does a stored door path cross a room?	Four-direction A* on the 0.10 m door grid	Three.js
How is a clicked point route found?	Four-direction A* on precomputed 0.01 m navigation tiles	Ollama
What is the shortest multi-room route?	Dijkstra over the door graph	SHACL
Does a route pass a rule?	SHACL over RDF measurements	The route colour itself
How is the 2D plan drawn?	SVG and JavaScript	A new route search
How is the wheelchair scene drawn?	Three.js/WebGL floor-slice rendering	IFCtoLBD
Who writes the explanation?	Ollama, using checked facts	A*

17 Important limits

- Bounding boxes are simpler than exact meshes. They are suitable for this visual route checker, but they approximate rotated or irregular shapes.
- Stored door routes use a 0.10 m grid. Interactive point routes use 0.01 m cells stored in compressed 5 m tiles. The finer grid improves click precision but needs more preprocessing, disk space and search work.
- The point grid expands solid obstacles by 0.445 m around the route centre. This represents a 0.90 m wheelchair envelope while retaining a 0.005 m geometry tolerance.
- The simulation is a floor-slice presentation, not a physics engine or a certified 1:1 wheelchair test.
- A clear route means it passed the encoded measurements and SHACL rules. It does not replace professional accessibility review or local legal approval.

- Poor IFC relationships or missing geometry can limit what the program can calculate. The audit reports missing geometry instead of inventing it.

18 Source map for technical readers

File	Main responsibility
`preprocess.py`	runs the complete preprocessing job
`backend/geometry.py`	extracts IFC geometry and element metadata
`backend/ifc_tools.py`	runs the pinned IFCtoLBD Java converter
`backend/routes.py`	space boundaries, grid, A*, measurements, Dijkstra
`backend/navigation.py`	0.01 m floor tiles, point-route A*, candidate path and final audit
`backend/shacl_runner.py`	runs pySHACL and maps rule results
`backend/package_writer.py`	writes browser-ready JSON and indexes
`backend/glb_export.py`	writes the compact GLB building model
`backend/short_explainer.py`	sends checked facts to Ollama
`server.py`	serves files and APIs; manages point requests, Ollama and verified project services
`frontend/app.js`	2D SVG, Three.js views, point selection, simulation and controls
`rules/ SHACL file`	accessibility SHACL constraints

19 Short recap

The route itself begins with IFC geometry, not with Ollama. Preprocessing builds stored door routes and compressed point-navigation tiles. A* finds orthogonal paths: between doors on the 0.10 m grid, or between two selected points on the 0.01 m tiles. Dijkstra joins passing door edges into shortest routes across the door graph. SHACL checks the stored measurements. The browser draws the backend coordinates in 2D and 2.5D, and the wheelchair follows the same point-route response. Ollama explains checked facts only after the deterministic calculation is complete.